

Análisis y Limpieza de Datos de Ventas

Proyecto de Portafolio – Miguel Ángel Palomera

Python | Pandas | Google Colab

1. Carga del dataset

```

import pandas as pd

df= pd.read_csv("/content/Copia de Data e_commerce.csv")
df=pd.DataFrame(df)
df.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	City	...	Postal Code	Region	Pr
0	4102	US-2021-102288	19/6/2021	23/6/2021	Standard Class	ZC-21910	Zuschuss Carroll	Consumer	United States	Houston	...	77095.0	Central	OF 100
1	9293	CA-2021-124114	2/3/2021	2/3/2021	Same Day	RS-19765	Roland Schwarz	Corporate	United States	Waco	...	76706.0	Central	O 100
2	8659	CA-2020-168361	21/6/2020	25/6/2020	Standard Class	KB-16600	Ken Brennan	Corporate	United States	Chicago	...	60623.0	Central	O 100
3	4712	CA-2018-112403	31/3/2018	31/3/2018	Same Day	JO-15280	Jas O'Carroll	Consumer	United States	Philadelphia	...	19120.0	East	O 100
4	2107	US-2018-152723	26/9/2018	26/9/2018	Same Day	HG-14965	Henry Goldwyn	Corporate	United States	Mesquite	...	75150.0	Central	O 100

5 rows × 21 columns

```

df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Row ID                9994 non-null  int64
1   Order ID              9994 non-null  object
2   Order Date            9994 non-null  object
3   Ship Date             9994 non-null  object
4   Ship Mode             9994 non-null  object
5   Customer ID           9994 non-null  object
6   Customer Name         9994 non-null  object
7   Segment               9994 non-null  object
8   Country/Region        9994 non-null  object
9   City                  9994 non-null  object
10  State                 9994 non-null  object
11  Postal Code           9983 non-null  float64
12  Region                9994 non-null  object
13  Product ID            9994 non-null  object
14  Category              9994 non-null  object
15  Sub-Category          9994 non-null  object
16  Product Name          9994 non-null  object
17  Sales                  9994 non-null  float64
18  Quantity              9994 non-null  int64
19  Discount              9994 non-null  float64
20  Profit                9994 non-null  float64
dtypes: float64(4), int64(2), object(15)
memory usage: 1.6+ MB
```

2. Análisis exploratorio inicial

Dimensiones
df.shape

(9994, 21)

Resumen Estadístico
df.describe()

	Row ID	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9983.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	4997.500000	55245.233297	229.858001	3.789574	0.156203	28.656896
std	2885.163629	32038.715955	623.245101	2.225110	0.206452	234.260108
min	1.000000	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	2499.250000	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	4997.500000	57103.000000	54.490000	3.000000	0.200000	8.666500
75%	7495.750000	90008.000000	209.940000	5.000000	0.200000	29.364000
max	9994.000000	99301.000000	22638.480000	14.000000	0.800000	8399.976000

#Conteo de nulos
df.isnull().sum().sort_values(ascending=False)

	0
Postal Code	11
Order ID	0
Order Date	0
Ship Date	0
Row ID	0
Ship Mode	0
Customer ID	0
Segment	0
Customer Name	0
Country/Region	0
City	0
State	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0

dtype: int64

Búsqueda de valores únicos de una columna/lista en específico mediante la función .unique
pd.Series(df['Ship Mode'].unique()).sort_values(ascending=False)

```

0
0 Standard Class
3 Second Class
1 Same Day
2 First Class

```

`dtype: object`

```

# Verificación de cuántas veces aparece cada tipo para validar importancia real en el negocio
df["Ship Mode"].value_counts()

```

```

count
Ship Mode
Standard Class 5968
Second Class 1945
First Class 1538
Same Day 543

```

`dtype: int64`

Comienza a programar o generar con IA.

```

# Conversión a porcentajes
df["Ship Mode"].value_counts(normalize=True).mul(100).round(2).astype(str) + '%'

```

```

proportion
Ship Mode
Standard Class 59.72%
Second Class 19.46%
First Class 15.39%
Same Day 5.43%

```

`dtype: object`

El análisis de los modos de envío ****evidencia que el 59.72% ****de los pedidos se procesan bajo la **modalidad Standard Class**, lo que indica que la operación logística está claramente orientada a la optimización de costos por encima de la velocidad de entrega. Esto sugiere una estrategia conservadora enfocada en eficiencia operativa.

Por otro lado, los servicios acelerados (**Second Class con 19.46% y First Class con 15.39%**) representan en conjunto **más del 34% de los envíos**, lo que confirma la existencia de un segmento de clientes dispuesto a pagar por rapidez, abriendo oportunidades de optimización en estrategias comerciales y paquetes premium de servicio.

Finalmente, el bajo uso de la modalidad **Same Day (5.43%)** indica una **adopción limitada del servicio exprés**, lo cual puede estar asociado a costos elevados, restricciones de cobertura o bajo posicionamiento comercial.

Esto representa una oportunidad para analizar la viabilidad del servicio, ajustar su propuesta de valor o rediseñar la estrategia de promoción.

3. Limpieza y transformación de datos

Conversión de fechas

```

df["Order Date"] = pd.to_datetime(df["Order Date"], dayfirst=True)
df["Ship Date"] = pd.to_datetime(df["Ship Date"], dayfirst=True)

```

```

df[["Order Date", "Ship Date"]].dtypes

```

```

0
Order Date  datetime64[ns]
Ship Date   datetime64[ns]

dtype: object

```

```
df[["Order Date", "Ship Date"]].head()
```

	Order Date	Ship Date
0	2021-06-19	2021-06-23
1	2021-03-02	2021-03-02
2	2020-06-21	2020-06-25
3	2018-03-31	2018-03-31
4	2018-09-26	2018-09-26

✓ Eliminación de duplicados

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df = df.drop_duplicates()
```

Se identificaron y eliminaron registros duplicados para garantizar la integridad de la información.

✓ Limpieza del Postal Code

```
df["Postal Code"] = df["Postal Code"].astype("Int64").astype(str)
df["Postal Code"] = df["Postal Code"].replace("<NA>", "Unknown")

```

```
df["Postal Code"].isna().sum()
```

```
np.int64(0)
```

Se identificaron y reemplazaron registros sin código postal o desconocidos para garantizar la integridad de la información

✓ Limpieza de Texto

```

# Evitar errores silenciosos en catálogo
text_cols = ['Ship Mode', 'Segment', 'Country/Region', 'State',
             'City', 'Region', 'Category', 'Sub-Category', 'Product Name'] # Columnas que contienen datos categóricos o nombres

df[text_cols] = df[text_cols].apply(lambda x: x.str.strip()) # Con función lambda se eliminan espacios invisibles al inicio y a

```

```
df.head()
```

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country/Region	City	...	Postal Code	Region	Product ID
0	4102	US-2021-102288	2021-06-19	2021-06-23	Standard Class	ZC-21910	Zuschuss Carroll	Consumer	United States	Houston	...	77095	Central	OFF-AP-10002906
1	9293	CA-2021-124114	2021-03-02	2021-03-02	Same Day	RS-19765	Roland Schwarz	Corporate	United States	Waco	...	76706	Central	OFF-BI-10004022
2	8659	CA-2020-168361	2020-06-21	2020-06-25	Standard Class	KB-16600	Ken Brennan	Corporate	United States	Chicago	...	60623	Central	OFF-BI-10003727
3	4712	CA-2018-112403	2018-03-31	2018-03-31	Same Day	JO-15280	Jas O'Carroll	Consumer	United States	Philadelphia	...	19120	East	OFF-BI-10003529
4	2107	US-2018-152723	2018-09-26	2018-09-26	Same Day	HG-14965	Henry Goldwyn	Corporate	United States	Mesquite	...	75150	Central	OFF-BI-10003460
5 rows × 21 columns														

Creación de columna nueva para tiempo de entrega

```
# Se crea variable para saber los días que tarda un pedido en entregarse
df['Delivery Days'] = (df['Ship Date'] - df['Order Date']).dt.days # dt.days Convierte # entero de días
```

```
# SE muestran los resultados de la nueva columna creada
df['Delivery Days'].describe().round(2)
```

Delivery Days	
count	9994.00
mean	3.96
std	1.75
min	0.00
25%	3.00
50%	4.00
75%	5.00
max	7.00

dtype: float64

```
# Promedio de entregas
df["Delivery Days"].mean().round(2)
```

np.float64(3.96)

```
#Por tipo de envío
df.groupby("Ship Mode")["Delivery Days"].mean().round(2)
```

Delivery Days	
Ship Mode	
First Class	2.18
Same Day	0.04
Second Class	3.24
Standard Class	5.01

dtype: float64

```
# Analisis de desempeño de entregas de menor a mayor tiempo
df.groupby('Ship Mode')['Delivery Days'].mean().sort_values().round(2)
```

Delivery Days	
Ship Mode	
Same Day	0.04
First Class	2.18
Second Class	3.24
Standard Class	5.01

dtype: float64

```
# Y por región
df.groupby("Region")["Delivery Days"].mean().round(2)
```

Delivery Days	
Region	
Central	4.06
East	3.91
South	3.96
West	3.93

dtype: float64

Generé la variable **Delivery Days** calculando la diferencia entre las fechas de pedido y envío, permitiendo medir así, los tiempos reales de despacho del pedido y analizando el desempeño logístico por modalidad, región y segmento. Obteniendo un promedio cercano a los 4 días

4. Análisis de ventas

Ventas y rentabilidad general

```
total_sales = df["Sales"].sum() #Total vendido
total_profit = df["Profit"].sum() #Total de ganancias
total_sales, total_profit
```

```
(np.float64(2297200.8603), np.float64(286397.0217))
```

```
profit_margin = (total_profit / total_sales) * 100
round(profit_margin, 2)
```

```
np.float64(12.47)
```

```
print(f"Ventas totales: ${total_sales:,.2f}")
print(f"Utilidad total: ${total_profit:,.2f}")
print(f"Margen de utilidad: {profit_margin:.2f}%")
```

Ventas totales: \$2,297,200.86
Utilidad total: \$286,397.02
Margen de utilidad: 12.47%

```
#Rentabilidad por línea de negocio
(df.groupby('Category')['Profit'].sum() / df.groupby('Category')['Sales'].sum() * 100).round(2)
```

0	
Category	
Furniture	2.49
Office Supplies	17.04
Technology	17.40

dtype: float64

El análisis del margen por categoría revela que Technology y Office Supplies son las líneas más rentables, con márgenes superiores al 17%, mientras que Furniture presenta un margen crítico de 2.49%, indicando problemas de eficiencia o estructura de costos. Con estos resultados se sugieren oportunidades claras de optimización en precios, descuentos y logística para la categoría de mueble. Por otro lado se concluye que las otras dos categorías pueden considerarse pilares financieros del negocio.

✓ Ventas por categoría

```
ventas_categoria = df.groupby('Category')['Sales'].sum().sort_values(ascending=False)
ventas_categoria
```

Sales	
Category	
Technology	836154.0330
Furniture	741999.7953
Office Supplies	719047.0320

dtype: float64

✓ Rentabilidad por subcategorías

```
# Cálculo del margen de ganancia por subcategoría:
# 1) Se agregan las ventas y utilidades totales por subcategoría
# 2) Se calcula el margen porcentual (Profit / Sales)
# 3) Se ordenan los resultados de mayor a menor para identificar las subcategorías más rentables
sales_sc = df.groupby('Sub-Category')['Sales'].sum()
profit_sc = df.groupby('Sub-Category')['Profit'].sum()

margin_sc = (profit_sc / sales_sc * 100).round(2).sort_values(ascending=False)
margin_sc.head()
```

0	
Sub-Category	
Labels	44.42
Paper	43.39
Envelopes	42.27
Copiers	37.20
Fasteners	31.40

dtype: float64

El análisis de rentabilidad por subcategoría muestra que productos como Labels, Paper y Envelopes presentan márgenes superiores al 40%, convirtiéndose en los principales generadores de utilidad del negocio. Estas subcategorías, caracterizadas por alta rotación y bajos

costos operativos, representan oportunidades estratégicas para potenciar volumen sin comprometer la rentabilidad. En contraste, subcategorías con menor margen requieren una evaluación conjunta con su volumen de ventas para determinar su impacto real en el resultado financiero.

```
# Por ganancias
profit_subcat = df.groupby('Sub-Category')['Profit'].sum().sort_values(ascending=False)
profit_subcat
```

Profit	
Sub-Category	
Copiers	55617.8249
Phones	44515.7306
Accessories	41936.6357
Paper	34053.5693
Binders	30221.7633
Chairs	26590.1663
Storage	21278.8264
Appliances	18138.0054
Furnishings	13059.1436
Envelopes	6964.1767
Art	6527.7870
Labels	5546.2540
Machines	3384.7569
Fasteners	949.5182
Supplies	-1189.0995
Bookcases	-3472.5560
Tables	-17725.4811

dtype: float64

```
# Por Ventas
ventas_subcat = df.groupby('Sub-Category')['Sales'].sum().sort_values(ascending=False).round(2)
ventas_subcat.head()
```

Sales	
Sub-Category	
Phones	330007.05
Chairs	328449.10
Storage	223843.61
Tables	206965.53
Binders	203412.73

dtype: float64

```
# ¿Qué subcategorías venden más dinero?
# venden poco pero dejan mucho margen o venden mucho pero dejan poco
sales_sc.sort_values(ascending=False).round(2)
```

Sales	
Sub-Category	
Phones	330007.05
Chairs	328449.10
Storage	223843.61
Tables	206965.53
Binders	203412.73
Machines	189238.63
Accessories	167380.32
Copiers	149528.03
Bookcases	114880.00
Appliances	107532.16
Furnishings	91705.16
Paper	78479.21
Supplies	46673.54
Art	27118.79
Envelopes	16476.40
Labels	12486.31
Fasteners	3024.28

dtype: float64

El análisis cruzado entre volumen de ventas y margen por subcategoría evidencia dos grupos estratégicos: subcategorías de alto volumen como Phones y Chairs, que sostienen los ingresos, y subcategorías de bajo volumen pero alto margen como Labels, Paper y Envelopes, que son clave para la rentabilidad del negocio. Esta diferenciación permite orientar decisiones comerciales equilibrando crecimiento en ingresos y maximización de utilidad.

▼ Ventas por región

```
ventas_region = df.groupby('Region')['Sales'].sum().sort_values(ascending=False)
ventas_region
```

Sales	
Region	
West	725457.8245
East	678781.2400
Central	501239.8908
South	391721.9050

dtype: float64

Se analizaron indicadores clave como ventas totales, utilidad, margen promedio y desempeño por categoría y región. Asimismo, se evaluó el tiempo promedio de entrega por modalidad de envío, permitiendo identificar diferencias operativas relevantes.

Así mismo se puede observar que aunque Phones lidera en ventas, Copiers es la subcategoría más rentable, lo que sugiere un mayor margen por unidad. Esto abre oportunidades para priorizar su promoción o replicar su estrategia de pricing en otras líneas.

▼ 5. Visualización de resultados

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
sns.set_style("whitegrid")
```

Gráfica de Modods de Envío (% Ship Mode)

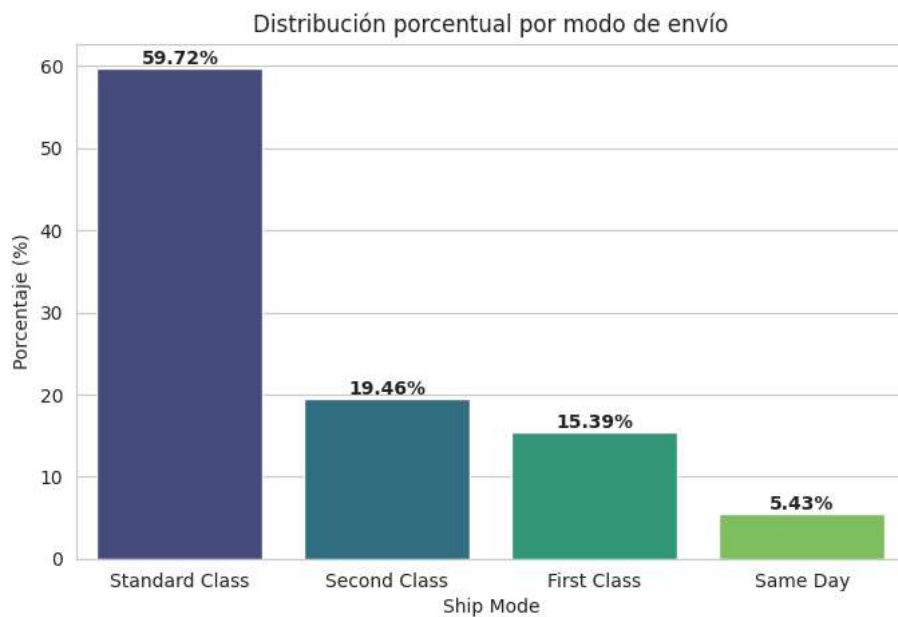
```
shipmode_pct = df['Ship Mode'].value_counts(normalize=True).mul(100).round(2)

plt.figure(figsize=(8,5))
sns.barplot(
    x=shipmode_pct.index,
    y=shipmode_pct.values,
    hue=shipmode_pct.index,      # Nueva función hue requerida
    legend=False,               # ocultar leyenda
    palette="viridis"
)

plt.title("Distribución porcentual por modo de envío")
plt.ylabel("Porcentaje (%)")
plt.xlabel("Ship Mode")

for i, v in enumerate(shipmode_pct.values):
    plt.text(i, v + 0.5, f"{v}%", ha='center', fontweight='bold')

plt.show()
```



Gráfica de Ventas por Categoría

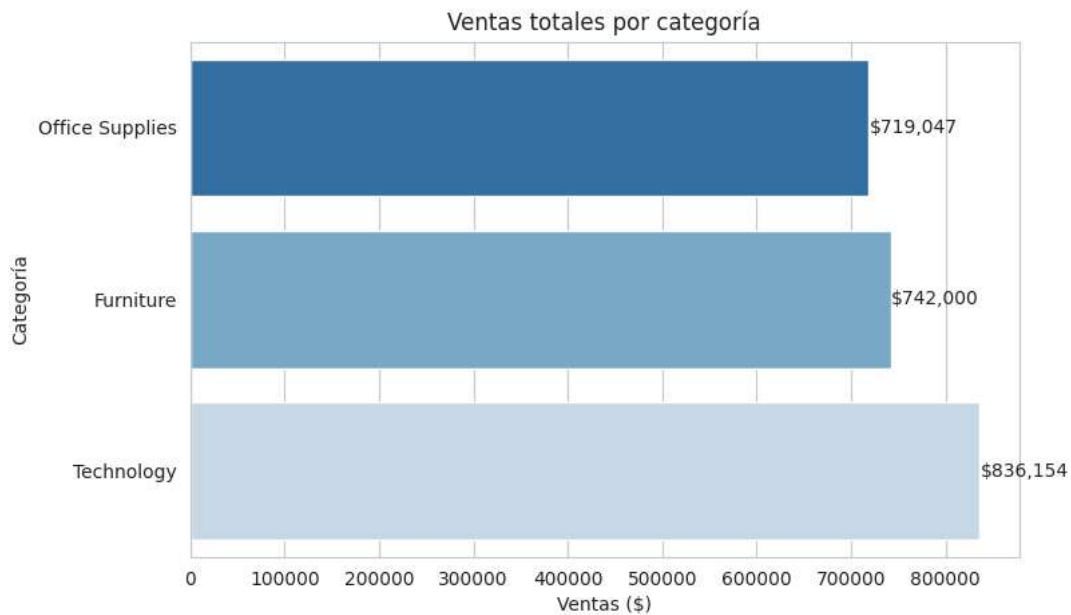
```
ventas_categoria = df.groupby('Category')['Sales'].sum().sort_values()

plt.figure(figsize=(8,5))
sns.barplot(
    x=ventas_categoria.values,
    y=ventas_categoria.index,
    hue=ventas_categoria.index,      # nuevo
    legend=False,
    palette="Blues_r"
)

plt.title("Ventas totales por categoría")
plt.xlabel("Ventas ($)")
plt.ylabel("Categoría")

for i, v in enumerate(ventas_categoria.values):
    plt.text(v + 1000, i, f"${v:,.0f}", va='center')

plt.show()
```



▼ Gráfica de Rentabilidad por Categoría

```
profit_margin = (
    df.groupby('Category')['Profit'].sum()
    / df.groupby('Category')['Sales'].sum()
) * 100

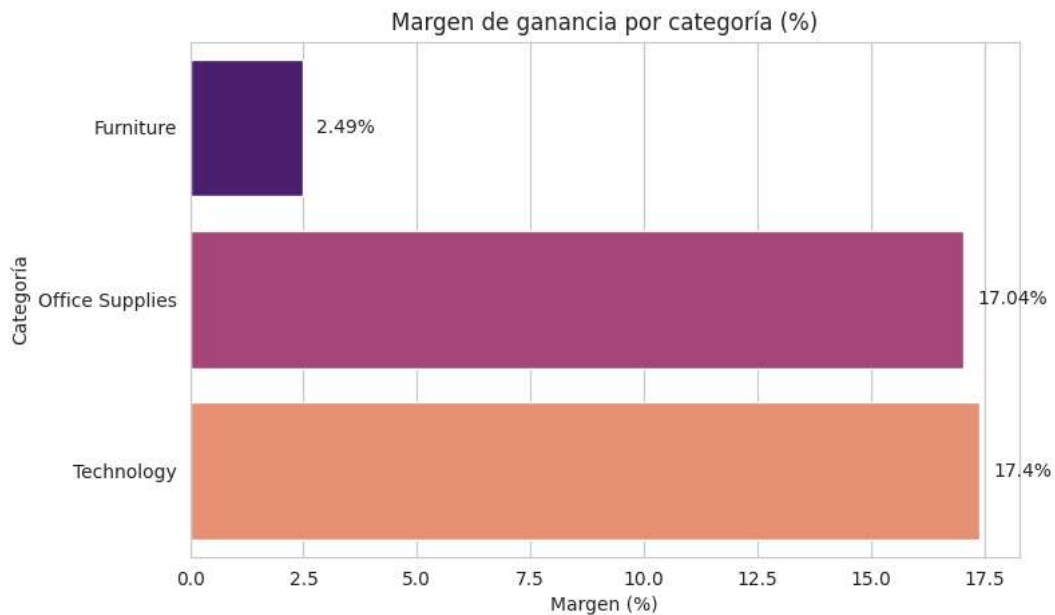
profit_margin = profit_margin.round(2)

plt.figure(figsize=(8,5))
sns.barplot(
    x=profit_margin.values,
    y=profit_margin.index,
    hue=profit_margin.index,
    legend=False,
    palette="magma"
)

plt.title("Margen de ganancia por categoría (%)")
plt.xlabel("Margen (%)")
plt.ylabel("Categoría")

for i, v in enumerate(profit_margin.values):
    plt.text(v + 0.3, i, f"{v}%", va='center')

plt.show()
```



Gráfica del Tiempo de Enregas por Modo de Envío

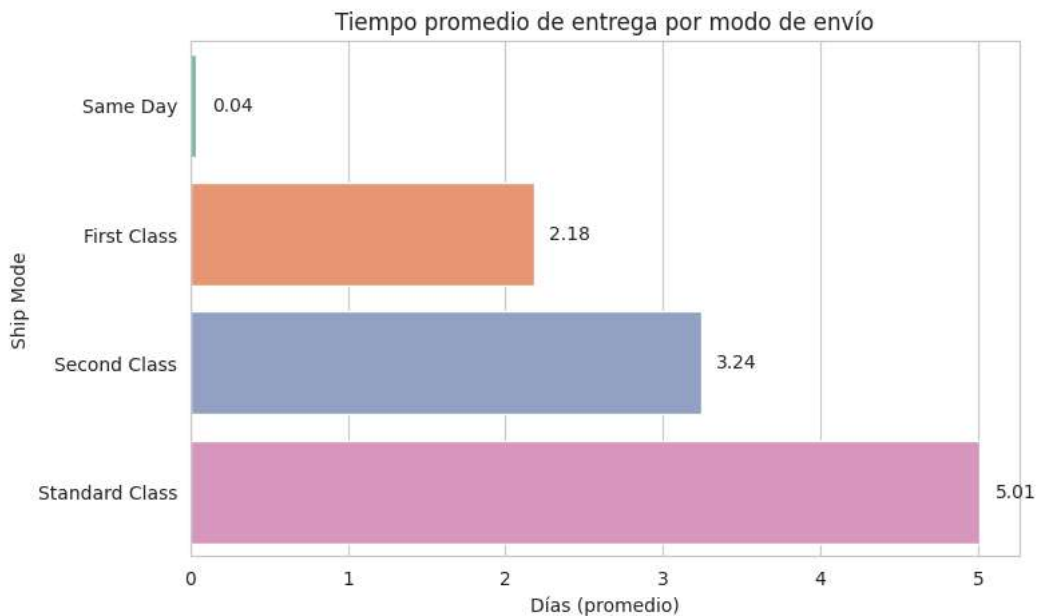
```
delivery_avg = df.groupby('Ship Mode')['Delivery Days'].mean().sort_values().round(2)

plt.figure(figsize=(8,5))
sns.barplot(
    x=delivery_avg.values,
    y=delivery_avg.index,
    hue=delivery_avg.index,
    legend=False,
    palette="Set2"
)

plt.title("Tiempo promedio de entrega por modo de envío")
plt.xlabel("Días (promedio)")
plt.ylabel("Ship Mode")

for i, v in enumerate(delivery_avg.values):
    plt.text(v + 0.1, i, v, va='center')

plt.show()
```



✓ Gráfica de las Ventas y la Rentabilidad General

```
# Ventas vs utilidad
kpis = pd.DataFrame({
    'Metrica': ['Ventas totales', 'Utilidad total'],
    'Valor': [total_sales, total_profit]
})

plt.figure(figsize=(7,5))
sns.barplot(
    data=kpis,
    x='Metrica',
    y='Valor',
    hue='Metrica',
    legend=False
)

plt.title('Comparación: Ventas vs Utilidad')
plt.ylabel('Monto ($)')
plt.xlabel('')

for i, v in enumerate(kpis['Valor']):
    plt.text(i, v, f"${v:,.0f}", ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()
```



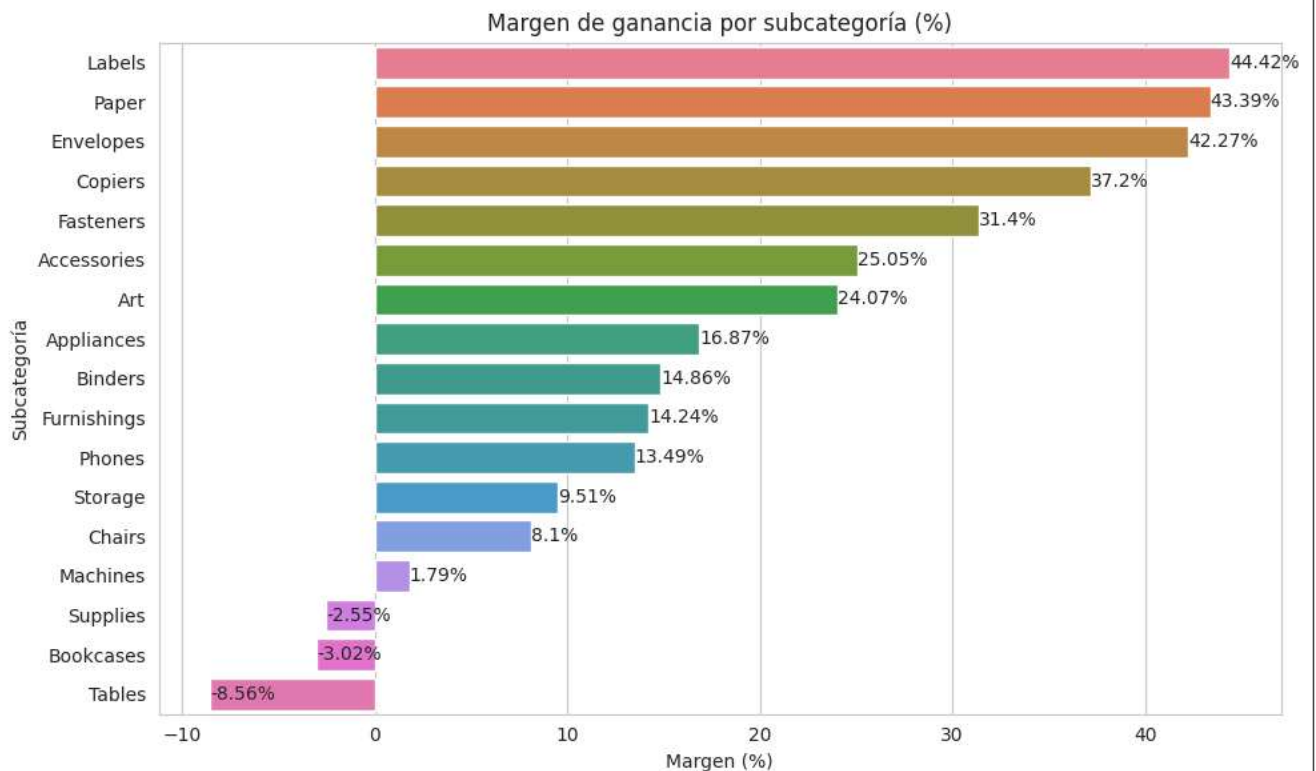
✓ Gráficas de Ventas y rentabilidad por Sub-categoría

```
#Gráfico de márgenes de ganancias por subcategoría (Rentabilidad)
plt.figure(figsize=(10,6))
sns.barplot(
    x=margin_sc.values,
    y=margin_sc.index,
    hue=margin_sc.index,
    legend=False
)

plt.title('Margen de ganancia por subcategoría (%)')
plt.xlabel('Margen (%)')
plt.ylabel('Subcategoría')

for i, v in enumerate(margin_sc.values):
    plt.text(v, i, f"{v}%", va='center', ha='left')

plt.tight_layout()
plt.show()
```

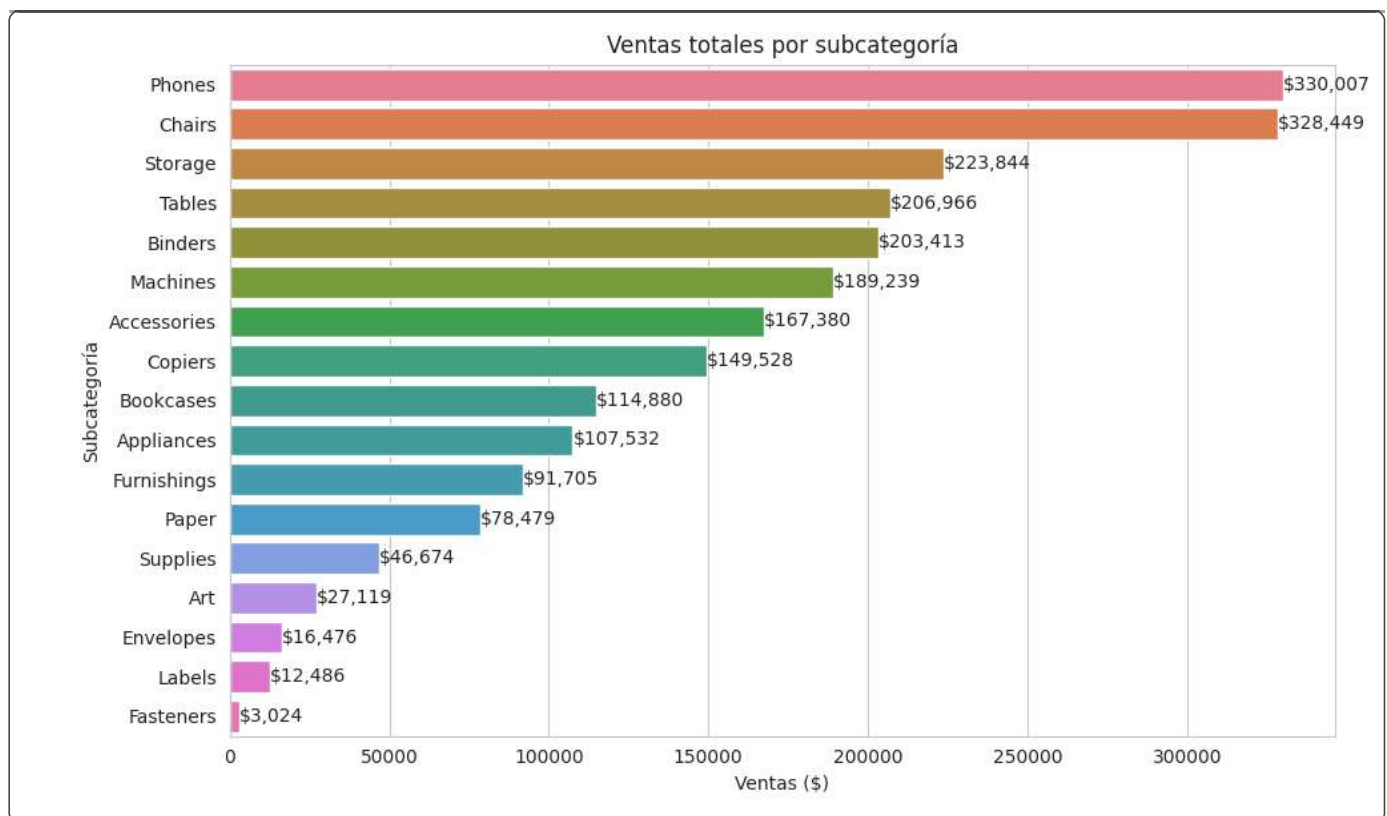


```
#Grafico de ventas por subcategoría
plt.figure(figsize=(10,6))
sns.barplot(
    x=ventas_subcat.values,
    y=ventas_subcat.index,
    hue=ventas_subcat.index,
    legend=False
)

plt.title('Ventas totales por subcategoría')
plt.xlabel('Ventas ($)')
plt.ylabel('Subcategoría')

for i, v in enumerate(ventas_subcat.values):
    plt.text(v, i, f"${v:,.0f}", va='center', ha='left')

plt.tight_layout()
plt.show()
```



Se generaron visualizaciones comparando ventas totales vs utilidad, así como análisis detallado por subcategoría, identificando tanto las líneas con mayor volumen de ventas como aquellas con mejor margen de rentabilidad. Esto permitió detectar subcategorías estratégicas y otras que requieren revisión de precios, descuentos o costos operativos.

6. Exportación del dataset limpio

Una vez completado el proceso de limpieza y transformación, se exporta una versión depurada del dataset para garantizar su reutilización en análisis posteriores y herramientas de visualización como Power BI o Tableau.

```
#Exportación del dataset limpio.
#Este archivo contiene los datos depurados y transformados, listos para su reutilización en análisis posteriores,
#visualización en herramientas BI o integración en pipelines de datos.
```

```
cols_finales = [
    'Order Date', 'Ship Date', 'Delivery Days',
    'Segment', 'Category', 'Sub-Category',
    'Sales', 'Profit', 'Ship Mode', 'Region'
]

df[cols_finales].to_csv(
    'ecommerce_clean_bi.csv',
    index=False,
    encoding='utf-8'
)
```

```
#Verificación de que el archivo efectivamente existe en el entorno
!ls
```

```
'Copia de Data e_commerce.csv'  ecommerce_clean_bi.csv  sample_data
```

```
#Descarga a local del nuevo archivo
```

```
from google.colab import files

files.download('ecommerce_clean_bi.csv')
```

```
# Para guardarlo en Drive

from google.colab import drive
drive.mount('/content/drive')

df.to_csv('/content/drive/MyDrive/ecommerce_clean_bi.csv', index=False)

Mounted at /content/drive
```

7. Conclusiones

Haz doble clic (o ingresa) para editar

Comportamiento de ventas y rentabilidad

Las subcategorías con mayor volumen de ventas fueron:

Phones – \$330,007

Chairs – \$328,449

Storage – \$223,844

Tables – \$206,966

Binders – \$203,413

Esto indica que los productos tecnológicos y de mobiliario concentran la mayor parte del ingreso total.

Sin embargo, alto volumen no siempre implica mayor rentabilidad. Al analizar las ganancias absolutas, las subcategorías más rentables fueron:

Copiers – \$55,617

Phones – \$44,516

Accessories – \$41,937

Destaca que Copiers, sin ser la subcategoría con más ventas, genera la mayor ganancia total, lo que sugiere precios más altos o mejores márgenes.

Análisis de márgenes de ganancia

El análisis de margen (% de ganancia sobre ventas) mostró que las subcategorías más eficientes fueron:

Labels – 44.42%

Paper – 43.39%
